

Arrays M Product of Array Except Self

Product of Array Except Self : Given array nums, return array output where output[i] equals product of all elements except nums[i] without using division, Example Input: [1,2,3,4] → Output: [24,12,8,6]; optimal approach uses prefix product (cumulative product before index) and suffix product (cumulative product after index) computed in two passes, Time Complexity $O(n)$, Space Complexity $O(1)$ extra excluding output array.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Prefix Sum/Product** - Understanding how to build cumulative products from left-to-right and right-to-left to avoid recomputation
- **Array Traversal** - Making multiple passes through an array to build intermediate results efficiently

4. Prefix & Suffix (Optimal)

Intuition

We can compute the product of all elements except the current one **without using extra prefix and suffix arrays**.

Instead, we reuse the result array and build the answer in two simple passes:

- In the **first pass**, we fill res[i] with the product of all elements to the left of i (prefix product).
- In the **second pass**, we multiply each res[i] with the product of all elements to the right of i (postfix product).

By maintaining two running values — prefix and postfix — we avoid the need for separate pref and suff arrays.

This gives us the same logic as the previous method, but with **$O(1)$ extra space**.

Algorithm

1. Initialize the result array res with all values set to 1.
2. Create a variable prefix = 1.
3. First pass (left to right):
 - For each index i:
 - Set res[i] = prefix (product of all elements to the left).
 - Update prefix *= nums[i].
4. Create a variable postfix = 1.
5. Second pass (right to left):
 - For each index i:
 - Multiply res[i] by postfix (product of all elements to the right).
 - Update postfix *= nums[i].
6. Return the result array res.

class Solution:

```
def productExceptSelf(self, nums: List[int]) -> List[int]:
    res = [1] * (len(nums))

    prefix = 1
    for i in range(len(nums)):
        res[i] = prefix
        prefix *= nums[i]
    postfix = 1
    for i in range(len(nums) - 1, -1, -1):
        res[i] *= postfix
        postfix *= nums[i]
    return res
```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity:
 - $O(1)O(1)$ extra space.
 - $O(n)O(n)$ space for the output array.

```
func productExceptSelf(nums []int) []int {
    res := make([]int, len(nums))
    for i := range res {
        res[i] = 1
    }

    prefix := 1
    for i := 0; i < len(nums); i++ {
        res[i] = prefix
        prefix *= nums[i]
    }

    postfix := 1
    for i := len(nums) - 1; i >= 0; i-- {
        res[i] *= postfix
    }
}
```

```

        postfix *= nums[i]
    }

    return res
}

```

Common Pitfalls

Using Division Without Handling Zeros

The division approach ($\text{totalProduct} / \text{nums}[i]$) fails when the array contains zeros. Dividing by zero causes runtime errors, and having multiple zeros requires special handling. Always count zeros first: with two or more zeros, the entire result is zeros; with exactly one zero, only the zero's position gets the product of other elements.

Off-by-One Errors in Prefix/Suffix Array Construction

When building prefix products, $\text{pref}[i]$ should contain the product of elements **before** index i , not including $\text{nums}[i]$. A common mistake is including $\text{nums}[i]$ in the prefix, which double-counts the element. The same applies to suffix arrays: $\text{suff}[i]$ should exclude $\text{nums}[i]$.

Integer Overflow with Large Products

When the array contains many large numbers, the product can overflow 32-bit integers. In languages with fixed-size integers, consider using long or BigInteger. The problem constraints usually prevent overflow, but edge cases with many elements near the maximum value should be tested.

Arrays E Is Subsequence

Is Subsequence : Given strings s and t , determine whether s is a subsequence (a sequence derived from another sequence by deleting some elements without changing order) of t ; Example Input: $s = \text{"abc"}$, $t = \text{"ahbgdc"}$ → Output: true; optimal approach uses Two Pointers (technique using two indices moving through sequences) advancing pointer in s only when characters match while scanning t once; Time Complexity $O(n)$; Space Complexity $O(1)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Two Pointers** - Traversing two sequences simultaneously with independent pointers
- **Recursion** - Breaking down the problem into smaller subproblems with base cases
- **Dynamic Programming** - Using memoization or tabulation to avoid redundant computations

4. Two Pointers

Intuition

The most efficient approach uses two pointers since we only need to make a single pass through both strings. Pointer i tracks our position in s , and pointer j tracks our position in t . We always advance j , but only advance i when we find a matching character. If we reach the end of s , all characters were found in order. This is optimal because each character in t is examined exactly once.

Algorithm

1. Initialize pointers $i = 0$ and $j = 0$.
2. While $i < \text{len}(s)$ and $j < \text{len}(t)$:
 - If $s[i] == t[j]$, increment i .
 - Always increment j .
3. Return $i == \text{len}(s)$.

class Solution:

```

def isSubsequence(self, s: str, t: str) -> bool:
    i = j = 0
    while i < len(s) and j < len(t):
        if s[i] == t[j]:
            i += 1
        j += 1
    return i == len(s)

```

Time & Space Complexity

- Time complexity: $O(n+m)O(n+m)$
- Space complexity: $O(1)O(1)$

Where nn is the length of the string ss and mm is the length of the string tt .

```

func isSubsequence(s string, t string) bool {
    i, j := 0, 0
    for i < len(s) && j < len(t) {
        if s[i] == t[j] {
            i++
        }
        j++
    }
    return i == len(s)
}

```

Common Pitfalls

Confusing Subsequence with Substring

A subsequence does not require consecutive characters, only that the order is preserved. For example, "ace" is a subsequence of "abcde", but "aec" is not because the order is violated. Make sure to only advance the pointer in s when characters match, not when looking for contiguous matches.

Forgetting to Handle Empty String Cases

When s is empty, it is always a subsequence of any string t (including an empty t). When t is empty but s is not, the answer is always false. Failing to handle these edge cases can lead to index-out-of-bounds errors or incorrect results.

Arrays M Best Time to Buy And Sell Stock II

Best Time to Buy And Sell Stock II : Given array prices where prices[i] is stock price on day i, find maximum profit with unlimited transactions (must sell before buying again), Example Input: [7,1,5,3,6,4] → Output: 7; optimal greedy approach sums all positive consecutive differences (if today's price > yesterday's price, add difference), Time Complexity O(n), Space Complexity O(1).

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Recursion** - Foundation for exploring all buy/sell decision combinations
- **Dynamic Programming (Memoization)** - Optimizes recursion by caching computed states
- **Dynamic Programming (Tabulation)** - Builds solutions iteratively from base cases
- **Greedy Algorithms** - Key insight that capturing every upward price movement is optimal

5. Greedy

Intuition

The key insight is that we can capture every upward price movement. If the price goes up from day i to day i+1, we can always "buy" on day i and "sell" on day i+1 to capture that profit. We do not need to track actual transactions because consecutive gains are equivalent to holding through multiple days. For example, buying at 1, holding through 3, 5, and selling at 6 gives the same profit as buying at 1, selling at 3, buying at 3, selling at 5, buying at 5, and selling at 6.

Algorithm

1. Initialize a profit variable to 0.
2. Iterate through the prices from day 1 to the last day.
3. If today's price is higher than yesterday's price, add the difference to profit.
4. Return the total profit.

class Solution:

```
def maxProfit(self, prices: List[int]) -> int:
    profit = 0

    for i in range(1, len(prices)):
        if prices[i] > prices[i - 1]:
            profit += (prices[i] - prices[i - 1])

    return profit
```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(1)O(1)$

```
func maxProfit(prices []int) int {
    profit := 0
    for i := 1; i < len(prices); i++ {
        if prices[i] > prices[i-1] {
            profit += prices[i] - prices[i-1]
        }
    }
    return profit
}
```

Common Pitfalls

Confusing with Best Time to Buy and Sell Stock I

This problem allows unlimited transactions, while Stock I only allows one. Using the Stock I approach (tracking min price and max difference) will undercount the total profit.

Missing Consecutive Gains

The greedy approach works because consecutive daily gains are equivalent to one larger transaction. Buying at 1, selling at 5 equals buying at 1, selling at 3, buying at 3, selling at 5. Missing this insight leads to overcomplicating the solution.

Adding Negative Profits

When using the greedy approach, only add the difference when prices[i] > prices[i-1]. Adding negative differences (price drops) reduces your profit incorrectly.

Wrong: adds negative changes too

```
profit += prices[i] - prices[i - 1]
```

```
# Correct: only add positive gains
if prices[i] > prices[i - 1]:
    profit += prices[i] - prices[i - 1]
```

Off-by-One in Loop

When comparing consecutive days, start the loop at index 1 (not 0) to safely access prices[i-1]. Starting at 0 causes an index out of bounds error.

Arrays E Can Place Flowers

Can Place Flowers : Given binary array flowerbed (0 empty, 1 occupied) and integer n, determine if n flowers can be planted without adjacent 1s (adjacency constraint means no two 1s next to each other); Example Input: flowerbed=[1,0,0,0,1], n=1 → Output: true; optimal approach greedily scans and plants when neighbors are 0; Time Complexity O(n); Space Complexity O(1).

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Arrays** - Traversing arrays and checking adjacent elements
- **Greedy Algorithms** - Making locally optimal choices (planting when possible) to achieve the best result
- **Boundary Handling** - Managing edge cases at the start and end of arrays

1. Iteration - I

Intuition

A flower can be planted at position i only if positions i-1, i, and i+1 are all empty. To handle edge cases at the boundaries, we pad the flowerbed with zeros at both ends. This way, we can apply the same rule uniformly across all positions without special boundary checks.

Algorithm

1. Create a new array with a 0 prepended and appended to the original flowerbed.
2. Iterate through the padded array from index 1 to length-2 (the original positions).
3. For each position, check if the current cell and both neighbors are empty (all zeros).
4. If so, plant a flower by setting the current cell to 1, and decrement n.
5. After processing all positions, return true if n is 0 or less (we placed enough flowers).

class Solution:

```
def canPlaceFlowers(self, flowerbed: List[int], n: int) -> bool:
    f = [0] + flowerbed + [0]

    for i in range(1, len(f) - 1):
        if f[i - 1] == 0 and f[i] == 0 and f[i + 1] == 0:
            f[i] = 1
            n -= 1

    return n <= 0
```

Time & Space Complexity

- Time complexity: $O(n)$
- Space complexity: $O(n)$

```
func canPlaceFlowers(flowerbed []int, n int) bool {
    f := make([]int, 0, len(flowerbed)+2)
    f = append(f, 0)
    f = append(f, flowerbed...)
    f = append(f, 0)

    for i := 1; i < len(f)-1; i++ {
        if f[i-1] == 0 && f[i] == 0 && f[i+1] == 0 {
            f[i] = 1
            n--
        }
    }

    return n <= 0
}
```

Common Pitfalls

Forgetting to Mark Planted Flowers

After deciding to plant a flower at position i, you must update the flowerbed to reflect the new flower. Otherwise, you may count overlapping positions as valid.

Wrong: not updating the flowerbed after planting

```
if f[i-1] == 0 and f[i] == 0 and f[i+1] == 0:
    n -= 1    # Missing: f[i] = 1
```

Off-by-One Errors at Boundaries

When not padding the array, forgetting to handle the first and last positions specially leads to index-out-of-bounds errors or incorrect neighbor checks.

Arrays E Find Pivot Index

Find Pivot Index : Given array nums, find index where sum of elements to left equals sum of elements to right; Example Input: [1,7,3,6,5,6] → Output: 3; optimal approach computes total sum then iterates maintaining running left sum; Time Complexity $O(n)$; Space Complexity $O(1)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Prefix Sum** - Computing cumulative sums to efficiently query range sums in $O(1)$ time
- **Array Traversal** - Iterating through arrays while maintaining running totals

3. Prefix Sum (Optimal)

Intuition

We can eliminate the need for a separate prefix sum array by maintaining a running left sum and computing the right sum on the fly. First, calculate the total sum of the array. As we iterate, the right sum at any index equals $\text{total} - \text{leftSum} - \text{nums}[i]$. We update leftSum after each comparison, keeping space usage constant.

Algorithm

1. Compute the total sum of the array.
2. Initialize leftSum = 0.
3. For each index i:
 - Compute $\text{rightSum} = \text{total} - \text{leftSum} - \text{nums}[i]$.
 - If $\text{leftSum} == \text{rightSum}$, return i.
 - Add $\text{nums}[i]$ to leftSum.
4. If no pivot is found, return -1.

class Solution:

```
def pivotIndex(self, nums: List[int]) -> int:
    total = sum(nums)
    leftSum = 0
    for i in range(len(nums)):
        rightSum = total - nums[i] - leftSum
        if leftSum == rightSum:
            return i
        leftSum += nums[i]
    return -1
```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(1)O(1)$

func pivotIndex(nums []int) int {

```
    total := 0
    for _, num := range nums {
        total += num
    }

    leftSum := 0
    for i := 0; i < len(nums); i++ {
        rightSum := total - leftSum - nums[i]
        if leftSum == rightSum {
            return i
        }
        leftSum += nums[i]
    }
    return -1
}
```

Common Pitfalls

Confusing Empty Sum as Zero

When the pivot is at index 0, the left sum is 0 (sum of no elements). Similarly, when the pivot is at the last index, the right sum is 0. Some solutions incorrectly skip checking the first or last index, assuming a pivot must have elements on both sides. The problem defines sums of empty ranges as 0, so boundary indices are valid pivot candidates.

Including the Pivot Element in the Sum

The pivot index itself should not be included in either the left sum or the right sum. A common mistake is computing leftSum as the sum up to and including index i, or computing rightSum starting from index i. The correct formula is $\text{rightSum} = \text{total} - \text{leftSum} - \text{nums}[i]$, which explicitly excludes the pivot element from both sums.

Arrays E Length of Last Word

Length of Last Word : Given a string consisting of words separated by spaces (whitespace characters), return length of the last word (a maximal substring containing non-space characters); Example Input: `s = "Hello World "` → Output: 5; optimal approach scans backward ignoring trailing spaces then counts characters until next space; Time Complexity $O(n)$; Space Complexity $O(1)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **String Traversal** - Iterating through characters in a string from either direction
- **Two-Pointer Technique** - Using index variables to track positions while scanning
- **Edge Case Handling** - Managing trailing spaces and multiple consecutive spaces in strings

2. Iteration - II

Intuition

Scanning from the end is more direct since we only care about the last word. First skip any trailing spaces, then count characters until we hit a space or the beginning of the string. This avoids processing earlier parts of the string entirely.

Algorithm

1. Start from the last index of the string.
2. Skip all trailing spaces by decrementing the index.
3. Count non-space characters by decrementing the index and incrementing length until a space is found or we reach the start.
4. Return length.

class Solution:

```
def lengthOfLastWord(self, s: str) -> int:
    i, length = len(s) - 1, 0
    while s[i] == ' ':
        i -= 1
    while i >= 0 and s[i] != ' ':
        i -= 1
        length += 1
    return length
```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(1)O(1)$

```
func lengthOfLastWord(s string) int {
    i, length := len(s)-1, 0
    for i >= 0 && s[i] == ' ' {
        i--
    }
    for i >= 0 && s[i] != ' ' {
        i--
        length++
    }
    return length
}
```

Common Pitfalls

Not Handling Trailing Spaces

Strings like "Hello World " have trailing spaces after the last word. If you simply scan from the end without first skipping spaces, you will count zero characters. Always skip trailing spaces before counting the last word's length.

Assuming Single Spaces Between Words

The input can have multiple consecutive spaces between words or at the beginning and end. Code that assumes exactly one space between words may produce incorrect results or fail on edge cases like " fly me to the moon ".

Arrays E Longest Common Prefix

Longest Common Prefix : Given an array of strings, find the longest prefix (substring starting at index 0) common to all strings; Example Input: `["flower", "flow", "flight"]` → Output: "fl"; optimal approach compares characters column-wise across all strings until mismatch occurs; Time Complexity $O(n*m)$ where m is prefix length; Space Complexity $O(1)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **String Manipulation** - Comparing characters, extracting substrings, and iterating through strings

- **Sorting** - Understanding lexicographic string ordering for the sorting approach
- **Trie Data Structure** - Building and traversing a prefix tree for the optimal solution

2. Vertical Scanning

Intuition

Instead of comparing entire strings horizontally, we can compare characters column by column across all strings. Check if all strings have the same character at position 0, then position 1, and so on. The moment we find a mismatch or reach the end of any string, we've found where the common prefix ends.

Algorithm

1. Iterate through character positions starting from index 0.
2. At each position i , check the character in the first string.
3. Compare this character against position i in every other string.
4. If any string is too short or has a different character, return the prefix up to index $i-1$.
5. If we complete the loop without returning, the entire first string is the common prefix.

class Solution:

```
def longestCommonPrefix(self, strs: List[str]) -> str:
    for i in range(len(strs[0])):
        for s in strs:
            if i == len(s) or s[i] != strs[0][i]:
                return s[:i]
    return strs[0]
```

Time & Space Complexity

- Time complexity: $O(n*m)O(n*m)$
- Space complexity: $O(1)O(1)$ since we did not use extra space.

Where nn is the length of the shortest string and mm is the number of strings.

```
func longestCommonPrefix(strs []string) string {
    for i := 0; i < len(strs[0]); i++ {
        for _, s := range strs {
            if i == len(s) || s[i] != strs[0][i] {
                return s[:i]
            }
        }
    }
    return strs[0]
}
```

Common Pitfalls

Not Handling Empty Strings in the Array

If any string in the input array is empty, the longest common prefix must be an empty string. Failing to check for this case before accessing characters can lead to index out of bounds errors.

Accessing Characters Beyond String Length

When comparing characters at a given index, you must ensure the index is valid for all strings being compared. A common mistake is to iterate based on one string's length without checking if shorter strings have characters at that position.

Arrays E Remove Element

Remove Element : Given array `nums` and integer `val`, remove all occurrences of `val` in-place (modify same memory without allocating new array) and return new length; Example Input: `nums=[3,2,2,3]`, `val=3` → Output: 2 with array modified to `[2,2]`; optimal approach uses two pointers overwriting valid elements; Time Complexity $O(n)$; Space Complexity $O(1)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Arrays** - Understanding how to traverse and modify array elements by index
- **Two Pointers Technique** - Using multiple pointers to track positions during array manipulation
- **In-Place Modification** - Modifying an array without using extra space

2. Two Pointers - I

Intuition

Instead of using extra space, we can overwrite unwanted elements in place.

We use a write pointer k that tracks where the next valid element should go.

As we scan through the array, whenever we find an element that is not equal to `val`, we write it at position k and move k forward.

At the end, everything before index k contains valid elements.

Algorithm

1. Initialize a pointer $k = 0$ to track the position for the next valid element.

2. Iterate through the array with index i:
 - If `nums[i]` is not equal to `val`, copy it to `nums[k]` and increment `k`.
3. Return `k` as the count of valid elements.

class Solution:

```
def removeElement(self, nums: List[int], val: int) -> int:
    k = 0
    for i in range(len(nums)):
        if nums[i] != val:
            nums[k] = nums[i]
            k += 1
    return k
```

Time & Space Complexity

- Time complexity: $O(n)$
- Space complexity: $O(1)$

func removeElement(nums []int, val int) int {

```
    k := 0
    for i := 0; i < len(nums); i++ {
        if nums[i] != val {
            nums[k] = nums[i]
            k++
        }
    }
    return k
}
```

Common Pitfalls

Returning the Wrong Value

A common mistake is returning the length of the original array instead of the count of remaining elements. The problem asks for the number of elements that are not equal to `val`, which may be less than the original array length. Always return the write pointer position (`k` or `n`) that tracks how many valid elements exist after removal.

Incrementing the Pointer After a Swap

When using the swap-with-end approach (Two Pointers - II), forgetting to recheck the current position after swapping leads to missed removals. After replacing `nums[i]` with the last element, the swapped element might also equal `val` and needs to be checked. Only increment `i` when the current element is valid; otherwise, decrement `n` and keep `i` unchanged.

Arrays M String Compression

String Compression : Given array of characters, compress in-place by replacing consecutive repeating characters with character followed by count (run-length encoding means representing consecutive duplicates as count), Example Input: `["a","a","b","b","c","c","c"]` → Output length 6 with array `["a","2","b","2","c","3"]`; optimal approach uses read pointer scanning and write pointer placing compressed form, Time Complexity $O(n)$, Space Complexity $O(1)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Two Pointers Technique** - Used to traverse and write to the array simultaneously while tracking consecutive characters
- **In-Place Array Modification** - Understanding how to modify an array without using additional space proportional to input size
- **String to Integer Conversion** - Converting count values to individual digit characters for multi-digit numbers

2. Two Pointers

Intuition

We can compress the array in-place using two pointers: one for reading (`i`) and one for writing (`k`). Since the compressed form is never longer than the original (a character followed by its count takes at most as much space as the repeated characters), we can safely overwrite the array as we go. This eliminates the need for extra space.

Algorithm

1. Initialize `k = 0` as the write pointer and `i = 0` as the read pointer.
2. While `i < n`, write `chars[i]` at position `k` and increment `k`.
3. Use pointer `j` starting at `i + 1` to find all consecutive characters equal to `chars[i]`.
4. If the count `j - i` exceeds 1, convert it to a string and write each digit to `chars[k++]`.
5. Move `i` to `j` and repeat.
6. Return `k` as the new length of the compressed array.

class Solution:

```
def compress(self, chars: List[str]) -> int:
    n = len(chars)
    k = i = 0
```



```

while i < n:
    chars[k] = chars[i]
    k += 1
    j = i + 1
    while j < n and chars[i] == chars[j]:
        j += 1

    if j - i > 1:
        for c in str(j - i):
            chars[k] = c
            k += 1

    i = j
return k

```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(1)O(1)$

```

func compress(chars []byte) int {
    n := len(chars)
    k, i := 0, 0

    for i < n {
        chars[k] = chars[i]
        k++
        j := i + 1
        for j < n && chars[i] == chars[j] {
            j++
        }

        if j-i > 1 {
            cnt := strconv.Itoa(j - i)
            for _, c := range cnt {
                chars[k] = byte(c)
                k++
            }
        }
        i = j
    }

    return k
}

```

Common Pitfalls

Forgetting to Write Multi-Digit Counts Character by Character

When the count of consecutive characters exceeds 9 (e.g., 12), you must write each digit separately ('1' then '2') rather than treating it as a single value. Failing to split the count into individual digit characters will produce incorrect output.

Writing Count for Single Characters

The problem specifies that you should only write the count if it is greater than 1. A common mistake is writing "a1" instead of just "a" for a single occurrence. Always check if (count > 1) before appending the count to the result.

Overwriting Unprocessed Characters in In-Place Solutions

When compressing in place, the write pointer can overwrite characters that haven't been read yet if not handled carefully. Since the compressed form is never longer than the original, this won't happen as long as you process groups completely before moving the write pointer forward.

Arrays M Reverse Words in a String

Reverse Words in a String: Given string with spaces, reverse order of words while removing extra spaces, approach split words or traverse from end and build result skipping multiple spaces, optimal time $O(n)$ because single pass, space $O(n)$ to store result string.

Example:

Input: " hello world "

Output: "world hello"

Arrays M Increasing Triplet Subsequence

Increasing Triplet Subsequence: Given array, determine if there exist indices $i < j < k$ such that $nums[i] < nums[j] < nums[k]$, approach greedily track smallest and second smallest values while scanning; if we find number larger than both, return true, optimal time $O(n)$, space $O(1)$.

Example:

Input: [2,1,5,0,4,6]

Output: true

Arrays M H-Index

H-Index: Given array citations where citations[i] = number of citations of ith paper, compute maximum h such that researcher has at least h papers cited at least h times, approach sort array descending and find largest index satisfying citations[i] ≥ i+1, optimal time O(n log n) for sorting, space O(1) if sorting in-place.

Example:

Input: [3,0,6,1,5]

Output: 3

Arrays E Kids With the Greatest Number of Candies

Kids With the Greatest Number of Candies: Given candies array and extraCandies, return boolean array where true if candies[i] + extraCandies ≥ maximum in array, approach first compute max then iterate and compare, optimal time O(n), space O(1) excluding output array.

Example:

Input: candies=[2,3,5,1,3], extra=3

Output: [true,true,true,false,true]

Arrays E Reverse Vowels of a String

Reverse Vowels of a String: Reverse only vowels (a,e,i,o,u both cases) in string, approach two pointers from both ends swapping vowels while skipping non-vowels, optimal time O(n), space O(n) if string converted to char array (since strings immutable in many languages).

Example:

Input: "hello"

Output: "holle"

Arrays E Find the Highest Altitude

Find the Highest Altitude: Given gain array where gain[i] represents net gain from point i to i+1, starting altitude 0, compute maximum altitude reached using prefix sum (running cumulative sum), optimal time O(n), space O(1).

Example:

Input: [-5,1,5,0,-7]

Output: 1
